# E2P: Exploração Inicial de Aplicações Web Orientada por Ações com Modelos de Linguagem Locais

**Henrique Oliveira da Cunha Franco**[1], **Humberto Torres Marques Neto**[1]

[1]Instituto de Informática – Pontifícia Universidade Católica de Minas Gerais (PUC Minas)

henrique.franco.1448652@sga.pucminas.br, humberto@pucminas.br

***Abstract.*** *This work investigates how far local language models can conduct initial exploration of an unfamiliar web application and observe potential failures. E2P connects project inspection, a bounded action catalogue, state observations, and test generation. Repeated explorations in two local configurations reached useful operations without a supplied navigation script or defect description. An audio player exception recurred across configurations, and browser checks distinguished it from unsupported defect hypotheses. Controlled checks show that property-specific assertions detect alterations, while a generated suite passed even when card creation was disabled. A reference function matrix organizes the evidence and exposes gaps between attempting, exercising, and verifying a function. The results characterize the exploratory reach of each configuration and the evidence needed to evaluate the resulting tests.*

***Resumo.*** *Este trabalho investiga até que ponto modelos de linguagem locais conseguem realizar uma exploração inicial de uma aplicação web desconhecida e observar possíveis falhas. O E2P conecta inspeção do projeto, catálogo limitado de ações, observações de estado e geração de testes. Explorações repetidas em duas configurações locais alcançaram operações úteis sem roteiro de navegação ou descrição de defeito fornecidos. Uma exceção no reprodutor de áudio ocorreu nas duas configurações, e verificações de navegador a distinguiram de hipóteses de defeito sem sustentação. Ensaios controlados mostram que asserções específicas detectam alterações, enquanto uma suíte gerada passou mesmo com a criação de cartões desativada. Uma matriz de funções de referência organiza as evidências e evidencia lacunas entre tentar, exercitar e verificar uma função. Os resultados caracterizam o alcance exploratório de cada configuração e a evidência necessária para avaliar os testes resultantes.*

## 1. Introdução

Testes ponta a ponta (*End-to-End*, E2E) exercitam jornadas de uso sobre um sistema integrado. Sua utilidade depende não apenas de executar interações, mas também de selecionar comportamentos relevantes e verificar seus resultados. Para uma aplicação web sem uma rotina de garantia de qualidade (*Quality Assurance*, QA), preparar essa automação envolve entender o produto, configurar seu ambiente, explorar suas funções, especificar casos e interpretar falhas. Automatizar somente a escrita de scripts deixa parte importante desse trabalho em aberto.

Modelos de linguagem de grande porte (*Large Language Models*, LLMs) têm sido investigados em geração de testes, reprodução de defeitos e apoio à manutenção de software [Wang et al. 2024, Kang et al. 2023]. Entretanto, uma resposta textual plausível não comprova que controles existem, que uma jornada foi executada ou que uma expectativa é correta. Estudos de agentes web evidenciam dificuldades na conclusão de tarefas realistas [Zhou et al. 2024, Drouin et al. 2024]. Em QA, essas dificuldades se somam ao risco de transformar um comportamento normal em relato de defeito.

Neste trabalho, *orientado por ações* caracteriza o papel do modelo na orquestração: a partir da observação da aplicação, ele escolhe uma operação em um catálogo explícito, executada e verificada por ferramentas. O protótipo Action E2E Prototype (E2P) integra um modelo existente a documentação e trechos de código, estado estruturado da interface, percepção visual opcional, ações limitadas e recursos de revisão humana. Seu propósito é apoiar a preparação de testes e a análise de evidências em aplicações web. A avaliação utiliza aplicações de pequeno porte com funções identificáveis, permitindo examinar como as decisões do modelo se relacionam com o comportamento observado.

O problema de pesquisa é: *até que ponto um modelo de IA consegue realizar uma exploração inicial de uma aplicação web desconhecida e, nesse processo, observar possíveis falhas?* O objetivo geral é investigar essa capacidade por meio do E2P, que conecta decisões do modelo à execução real da aplicação. Os objetivos específicos são: (i) reconhecer funções a partir de documentação, código e interface; (ii) selecionar e executar ações relacionadas ao propósito do produto; (iii) registrar estados e evidências que permitam revisar a trajetória; e (iv) analisar hipóteses de falha e os mecanismos utilizados para verificá-las.

A contribuição é o E2P, um instrumento local que torna rastreável o percurso entre decisão do modelo, ação executada, evidência e asserção. As trajetórias mostram operações iniciadas sem roteiro e estados nos quais a instrumentação captura falhas reproduzíveis. Os ensaios de navegador demonstram em que casos as asserções distinguem alterações controladas e preservam a expectativa aprovada, assim como a diferença entre um teste aprovado e uma função efetivamente verificada.

O restante do artigo apresenta a fundamentação e os trabalhos relacionados na Seção 2, a metodologia na Seção 3, a implementação na Seção 4, os resultados na Seção 5, as ameaças à validade na Seção 6 e as conclusões na Seção 7.

## 2. Referencial Teórico e Trabalhos Relacionados

### 2.1. Da execução de interações à avaliação de comportamento

Um caso de teste associa condições iniciais, ações e resultados esperados. Um oráculo é o critério utilizado para decidir se o resultado observado é aceitável. No E2P, critérios em linguagem natural antecedem a automação e podem ser revisados. Essa separação é necessária porque a validade sintática do script, a existência do seletor e a aprovação da execução não garantem correspondência com a intenção do caso.

A revisão de Wang et al. [Wang et al. 2024] organiza aplicações de LLMs em diferentes atividades de teste e destaca desafios de sua integração com outras técnicas. A distinção entre gerar entradas, produzir asserções e avaliar resultados orienta este estudo: uma melhoria em uma dessas atividades não é automaticamente transferível às demais.

Em particular, medir somente quantos testes passam favorece testes permissivos, que podem permanecer verdes mesmo diante de um defeito.

Na avaliação de hipóteses, distinguem-se três afirmações: uma interação ocorreu; seu efeito foi reproduzido; esse efeito viola uma expectativa justificável. As duas primeiras são observacionais. A terceira requer evidência normativa, como um requisito, uma mensagem explícita da interface ou uma regra de domínio sustentada pelo projeto. O código auxilia na explicação causal, mas não constitui sozinho uma especificação infalível: ele também pode conter o defeito investigado. Essa distinção fundamenta a análise dos falsos positivos apresentada adiante.

### 2.2. Agentes de interface e testes baseados em uso

WebArena oferece aplicações e tarefas web para avaliar conclusão funcional, em vez de apenas a plausibilidade do texto produzido, e compara agentes com participantes humanos [Zhou et al. 2024]. WorkArena e BrowserGym enfatizam ações padronizadas, observações multimodais e tarefas de software corporativo [Drouin et al. 2024]. Esses trabalhos fundamentam o uso de ambientes registráveis e medidas ligadas ao comportamento efetivamente realizado. O E2P investiga a iniciativa exploratória anterior à definição de uma tarefa de teste, preservando a trajetória que poderá fundamentá-la.

No domínio móvel, GPTDroid estrutura a exploração como interação iterativa entre modelo e aplicação, utilizando informações da interface, retorno da execução e memória de funcionalidades [Liu et al. 2024]. O E2P adota o ciclo de observar e agir em aplicações web, restringindo decisões a ações catalogadas. Sua memória de revisões humanas recupera informações entre execuções, complementando a trajetória da sessão atual.

Avgust transforma vídeos de uso em uma representação de estados para apoiar a geração de testes em aplicações móveis [Zhao et al. 2022]. A aproximação com o E2P está na valorização de jornadas observadas. A diferença é a origem dessas jornadas: no E2P, elas são produzidas pela exploração guiada pelo modelo, e não fornecidas previamente como vídeos de usuários. Isso reduz a necessidade de uma gravação inicial, mas transfere ao explorador a responsabilidade de alcançar comportamentos úteis.

### 2.3. Geração e verificação de requisitos

LIBRO gera testes de reprodução a partir de relatos de defeitos e utiliza pós-processamento para avaliar e ordenar candidatos [Kang et al. 2023]. Enquanto esse trabalho parte de um problema relatado, o E2P também propõe hipóteses a partir da interface explorada. Portanto, além da reprodução, precisa avaliar se a própria alegação de erro é válida.

Ferreira et al. [Ferreira et al. 2025] apresentam um estudo industrial com geração de cenários Gherkin a partir de histórias de usuário e conversão para testes Cypress usando HTML. Os cenários foram considerados úteis em 95% das avaliações; 60% dos testes puderam ser utilizados como gerados, enquanto os demais exigiram ajustes, nova geração ou descarte. Esse resultado apoia o uso de artefatos intermediários revisáveis. Diferentemente daquele estudo, o E2P precisa inferir oportunidades de teste de um projeto e de sua execução, sem pressupor um conjunto completo de histórias de usuário.

Panta combina análise estática de fluxo e cobertura dinâmica para orientar iterativamente a geração de testes unitários [Gu et al. 2026]. TestWeaver usa recortes de programa, testes relacionados e anotações de execução para fornecer contexto dirigido ao modelo [Le et al. 2026]. Essas abordagens motivam o uso de sinais verificáveis e de contexto selecionado. No E2P, a análise se concentra nas funções de interface exercitadas e em seus resultados observáveis.

GUISpector verifica requisitos em linguagem natural sobre protótipos de interface com um agente multimodal e disponibiliza resultados para supervisão [Kolthoff et al. 2025]. O E2P compartilha o interesse na rastreabilidade entre requisito e trajetória, mas também investiga a descoberta dos próprios casos. Isso torna a procedência da expectativa um problema central: inferir um requisito exige cautela adicional em relação a verificar um requisito fornecido.

SpecOps decompõe o teste de agentes de interface gráfica (*Graphical User Interface*, GUI) em geração, preparação, execução e validação [Ahmed et al. 2026]. Seu objeto são agentes de IA em ambientes reais, enquanto o E2P avalia aplicações web. A separação dos estágios orienta a arquitetura do protótipo e a identificação da origem de cada evidência.

### 2.4. Posicionamento do trabalho

O E2P se posiciona na investigação da exploração inicial de aplicações web por um modelo de IA orientado por ações. Diante de um projeto apresentado sem roteiro de teste, o modelo precisa reconhecer sua finalidade, identificar possibilidades de interação e escolher como obter informação sobre seu funcionamento. A questão central é quanto dessa atividade pode ser realizada de forma útil e que indícios de falha podem surgir durante o percurso.

Os trabalhos relacionados oferecem fundamentos para essa investigação: agentes de interface sustentam o ciclo de observar e agir; geração de cenários inspira os artefatos intermediários; e estudos de reprodução e avaliação orientam a conferência das hipóteses. Essas contribuições informam a arquitetura do E2P e os critérios de análise. O objeto empírico deste trabalho é a atuação do modelo dentro dessa implementação, conectada a uma aplicação executada e a evidências de suas próprias decisões.

## 3. Metodologia

### 3.1. Delineamento e questões de pesquisa

A pesquisa é aplicada e exploratória, centrada na atuação do modelo dentro do E2P. A inspeção do projeto e a observação da interface fornecem o contexto para escolher ações, acompanhar consequências e propor verificações e hipóteses.

As questões de pesquisa são: **QP1**, quais funcionalidades o modelo reconhece e consegue exercitar durante uma exploração inicial; **QP2**, quais evidências permitem identificar possíveis falhas da aplicação; **QP3**, como os mecanismos de validação do E2P preservam a relação entre ação, resultado e expectativa; e **QP4**, como as condições de execução e os recursos computacionais delimitam a profundidade alcançada. O foco está na instância atual do protótipo e nas atividades que ela torna executáveis.

A avaliação articula três fontes de evidência: caracterização das aplicações e inspeção de artefatos; explorações com configuração mantida fixa em cada rodada; e verificações de navegador sobre propriedades determinadas. Essa organização separa a iniciativa exploratória da capacidade de uma asserção detectar uma violação.

### 3.2. Aplicações utilizadas e comportamentos de interesse

Para observar decisões em interfaces com finalidades diferentes, foram utilizados os exemplos públicos de Vanilla Web Projects [Traversy 2026]. O conjunto reúne 21 aplicações: formulários, listas, buscas, jogos, reprodução de mídia e recursos de voz. A escolha permite relacionar cada ação a um propósito identificável e examinar tanto operações locais quanto comportamentos dependentes de tempo ou de serviços externos. O catálogo Dopa [da Cunha Franco 2026], desenvolvido pelo próprio autor deste TCC, foi utilizado como cenário inicial de integração do protótipo; os exemplos públicos oferecem os casos de estudo apresentados nesta avaliação.

A caracterização considera organização do código, entradas, estado e dependências. Cada exemplo possui um documento HTML de entrada e dois a quatro arquivos-fonte, com 118 a 372 linhas físicas não vazias de HTML, CSS e JavaScript, incluindo comentários. Dependências instaladas, arquivos gerados, mídia e arquivos minificados são excluídos da contagem. Os exemplos podem ser servidos estaticamente e não possuem serviço próprio no servidor. Cinco utilizam chamadas `fetch`, quatro utilizam armazenamento no navegador e dois empregam APIs de fala.

Essas características descrevem aplicações de pequeno porte e escopo funcional delimitado. O porte é sustentado pela estrutura concentrada no navegador e pela quantidade restrita de responsabilidades, além do volume de código. A exigência de teste, por sua vez, depende do comportamento: uma interface de poucos arquivos pode envolver cálculo, estado acumulado, áudio ou temporização. Por isso, a análise apresenta as funções exercitadas e os resultados observáveis, sem utilizar linhas de código como uma escala de dificuldade.

A Tabela 1 apresenta as aplicações por finalidade predominante. As famílias organizam a leitura dos casos; capacidades como persistência e dependência de rede podem aparecer em mais de uma família. O inventário complementar associa cada aplicação a seus arquivos, medidas, comportamentos de referência e artefatos de execução.

### 3.3. Exploração e coleta de evidências

Na exploração inicial, o modelo recebe documentação e trechos selecionados de código, estado estruturado da interface e, quando habilitadas, imagens da página. Uma aplicação é considerada desconhecida para a sessão quando seu projeto é apresentado sem sequência de passos ou descrição de defeito a procurar; a definição se refere ao contexto fornecido durante a avaliação. O E2P disponibiliza controles observados em um catálogo de ações. O modelo escolhe uma operação e seus parâmetros; depois da execução, uma nova observação registra a mudança e orienta a próxima decisão. A Figura 1 apresenta o processo.

A validação inclui duas explorações por aplicação em Expense Tracker, Movie Seat Booking e Modal Menu Slider, alternando os alvos entre repetições. Os casos permitem investigar entrada tipada e cálculo, seleção com persistência e ciclo de painéis,

**Tabela 1. Aplicações utilizadas e comportamentos de interesse para exploração.**

| Família (n) | Aplicações e finalidade | Exigência de observação |
|---|---|---|
| Dados e formulários (5) | DOM Array Methods: listas; Expense Tracker: lançamentos; Form Validator: campos; Memory Cards: cartões; Movie Seat Booking: assentos. | Entrada, validação, alteração de valores e estado preservado. |
| Busca e seleção (5) | Exchange Rate: moedas; Infinite Scroll Blog: publicações; Lyrics Search: letras; Meal Finder: receitas; Product Filtering: produtos. | Consulta, resultados, seleção de detalhe e resposta de serviços. |
| Mídia e voz (4) | Custom Video Player: vídeo; Music Player: áudio; Speak Number Guess: reconhecimento; Speech Text Reader: síntese. | Controles, identidade do conteúdo e sinais de reprodução ou fala. |
| Jogos e temporização (5) | Breakout: colisões; Hangman: palpites; New Year Countdown: calendário; Relaxer App: respiração; Typing Game: digitação. | Teclado, sequências, pontuação e mudanças temporais. |
| Interação e ordenação (2) | Modal Menu Slider: painéis; Sortable List: lista reordenável. | Abertura, fechamento e alteração da ordem. |

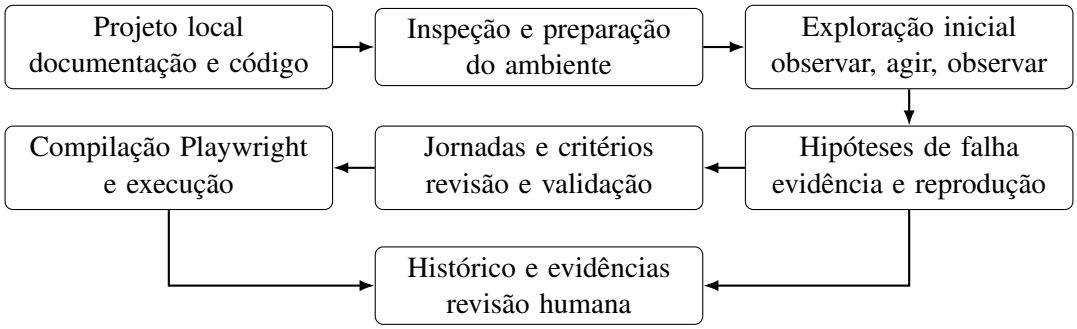



**Figura 1. Exploração inicial e transformação das observações em artefatos revisáveis.**

respectivamente. O perfil de acesso é convidado, com dados fictícios e contexto de navegador isolado em cada tentativa. A implementação dos serviços é identificada por resumos criptográficos (*hashes*) e mantida fixa durante a coleta.

As rodadas complementares utilizam Form Validator, Music Player, Memory Cards e TodoMVC React [TasteJS 2026], com duas repetições intercaladas por configuração. O último acrescenta outra origem, componentes React, gerenciamento de estado e filtros por rotas, sem serviço próprio no servidor. Foram usadas as mesmas revisões dos alvos nas duas configurações, com arquivos-fonte conferidos após normalização dos finais de linha.

O procedimento preserva ações, parâmetros, estados e diagnósticos. A matriz de referência associa funções a passos e estados. Na rodada em Apple Silicon, o protocolo registrou o resumo criptográfico da lista no início e não a forneceu ao modelo; sua aplicação à coleta anterior é retrospectiva. Uma tentativa registra a ação proposta; *exercitar* requer observar o efeito da operação, e *verificar* exige confrontá-lo com uma expectativa fundamentada. A classificação automática orienta a inspeção, pois a ação ou a asserção encontrada pode se referir a outra propriedade.

### 3.4. Ambiente de inferência e recursos disponíveis

O modelo local de 27 bilhões de parâmetros utiliza os pesos `Qwen3.8-27B-UD-IQ1_M.gguf`, quantização IQ1_M e projetor visual BF16, sob o nome de configuração `qwen3.8-vl-27b-iq1m-64k`. O servidor llama.cpp b10819 opera com Vulkan, uma posição de atendimento (*slot*) para uma requisição de inferência por vez, contexto configurado de 65.536 tokens, cache de chaves e valores (KV) em Q8 e *Flash Attention* [ggml-org 2026]. A máquina utiliza Windows, Ryzen 7 7800X3D, Radeon RX 9070 XT com 16 GB de memória gráfica e 32 GB de RAM. A exploração e os testes são executados por Playwright com Chromium.

A segunda configuração utiliza macOS, Apple M4 com 16 GB de memória unificada, Qwen3-VL de 4 bilhões de parâmetros em Q4_K_M, projetor visual Q8 e contexto reservado de 16.384 tokens. A inferência usa llama.cpp com Metal. A seleção de perfil considera memória recuperável pelo sistema e o limite de trabalho informado pelo servidor. A quantização e o contexto reservado influenciam o orçamento de memória de pesos, cache e processamento visual. O contexto efetivamente usado depende da etapa e das evidências selecionadas.

O orçamento da exploração é de até 20 ações e 180 segundos nas duas configurações. A profundidade depende das escolhas, do tempo de inferência, da resposta do alvo e do custo de organizar código, histórico e imagens. O relógio da página é isolado durante as decisões para reduzir a interferência da latência em interfaces temporizadas.

### 3.5. Validação e critérios de interpretação

A exploração é examinada por ações concluídas, funções reconhecidas, mudanças de estado e diagnósticos observados. A avaliação de um caso executável confronta a intenção do critério com sua asserção: entidade, valor esperado, comparação e momento de verificação. Um teste de persistência, por exemplo, precisa criar estado, recarregar e conferir o mesmo estado restaurado. A existência de um campo ou de um cabeçalho serve como evidência de interface, mas não substitui uma propriedade de domínio.

A implementação é submetida à suíte interna e a verificações de navegador com versões de referência e cópias modificadas dos alvos. São avaliadas mudança de faixa, atualização de saldo e armazenamento, além da preservação de um valor esperado em formulário de controle com regra explícita de normalização. Em cada caso, a mesma suíte é executada no original e na cópia alterada; o efeito da alteração é conferido separadamente no navegador. Esses ensaios verificam a resposta das asserções, enquanto as explorações examinam as decisões do modelo.

As ações associadas a exceções são repetidas em sessões limpas. A análise técnica assistida por IA confronta a ação, o efeito observado, o comportamento previsto no projeto e a reprodução. Na revisão da coleta em Apple Silicon, os registros foram conferidos por inspeção e por ensaios dirigidos no Chromium em Windows, sem nova chamada ao modelo. Nesses ensaios, repetiram-se interações de formulário, áudio, cartões e tarefas e reexecutou-se a suíte arquivada dos cartões em dois estados de implementação.

O repositório do E2P[1] registra configurações, arquivos-fonte, protocolos e

[1] `https://github.com/pieceofhell/action-e2e`. A revisão `f7df0df` identifica o suplemento da coleta no M4.

evidências. O comando `evaluate:corpus` reproduz o inventário estrutural; `evaluate:functions` refaz a classificação automática a partir das trajetórias. Os ensaios de navegador examinam as operações e propriedades separadamente.

## 4. Descrição da Implementação

### 4.1. Serviços e preparação do ambiente

O E2P utiliza Node.js e Express, interface HTML/CSS/JavaScript, Cheerio para inspeção estática e Playwright para observação e execução [Microsoft 2026]. Serviços separados tratam inspeção, ambiente, provedor, exploração, planejamento, geração, hipóteses e histórico. A inspeção seleciona documentação e trechos do repositório; o modelo contribui para sua interpretação, enquanto a execução depende de comandos e controles admitidos pelo instrumento. Essa separação torna possível localizar o estágio em que uma falha ocorreu.

O ambiente pode ser servido estaticamente, iniciado por comando, conectado a uma URL existente ou submetido apenas à análise. O comando e a URL inferidos são revisáveis. Cada execução mantém artefatos em diretório próprio, sem inserir testes no código do alvo. Instalar dependências e iniciar a aplicação ainda pode produzir efeitos no ambiente; a separação de diretórios não constitui isolamento completo.

O adaptador atende diferentes provedores; as explorações avaliadas usam llama.cpp diretamente. No modo convidado, capturas podem complementar o modelo de objetos do documento (*Document Object Model*, DOM) quando o provedor suporta visão. O envio ao modelo é controlado separadamente do registro da imagem. O modo autenticado é uma prova de conceito somente para leitura, com referências a segredos por variáveis de ambiente, sessões isoladas e quarentena de artefatos; vídeos e rastros ficam desabilitados nesse modo.

### 4.2. Exploração limitada e representação de estado

A cada observação, o explorador constrói um catálogo de controles com identificadores, nomes acessíveis e operações admitidas. O modelo escolhe nesse catálogo, e o executor resolve o controle correspondente. Respostas estruturadas em JSON e decisões inválidas recebem correção limitada. A política de participação obrigatória do modelo (*AI-first*) interrompe a progressão quando falta uma decisão válida, preservando a evidência parcial.

Objetivos QA derivados da inspeção contextualizam entrada, seleção, navegação, validação, temporização, persistência e cálculo. O perfil dos registros limita a exploração a até 20 ações e 180 segundos; esperas explícitas são limitadas a 12 segundos. Uma recarga depende de indícios de persistência e mudança anterior. Essas oportunidades orientam a descoberta, mas sua ocorrência não é uma asserção de correção.

O relógio da página convidada é pausado durante a inferência e retomado antes da ação, reduzindo mudanças silenciosas de estado enquanto o modelo decide. A inferência integra o orçamento total; serviços externos mantêm sua própria temporização. A política prioriza entradas fictícias ordinárias, com correção limitada após validação e respeito aos limites declarados pelos controles. O esgotamento do catálogo encerra a exploração das ações permitidas naquela trajetória.

### 4.3. Planejamento, compilação e execução

Fluxos e critérios devem remeter a controles e transições registrados. O planejamento recebe resumos de ações e estados com identificadores, reduzindo repetição de catálogos extensos no contexto. A revisão permite selecionar e editar critérios. Em lote, são admitidos fluxos fundamentados de confiança média/alta ou, no modo convidado, um teste explicitamente limitado de transição observada, desde que haja ação concluída, mudança, controle estável e resultado citado.

No caminho convidado, o gerador compila a jornada executada em Playwright, no modo `model-journey-compiled`. O modelo escolhe a trajetória e participa da definição de seu significado; o compilador determina a estrutura executável e as verificações disponíveis. A qualidade do teste depende, portanto, do comportamento alcançado e da preservação do critério na tradução para asserção.

Na instância avaliada, um contrato registra a origem da expectativa, a ação, a comparação, a evidência e a identificação da asserção. Critérios editados são revalidados; os não suportados permanecem não verificados. O contrato torna a tradução inspecionável. O executor distingue término operacional, falha de asserção e erro de infraestrutura. O sucesso exige relatório de execução válido, e medidas de cobertura exigem um conjunto de referência definido.

### 4.4. Hipóteses, reprodução e memória

Um papel autor propõe hipóteses a partir da trajetória; um papel crítico examina ação, expectativa e efeito alegado. Regras determinísticas filtram candidatos e podem registrar exceções do navegador. Os papéis utilizam o mesmo perfil de modelo. Hipóteses elegíveis passam por reprodução em contexto limpo, repetindo ações conhecidas e comparando o efeito. A procedência da expectativa é conferida como requisito adicional para classificar a violação.

O coletor registra diálogos nativos vinculados à ação, permitindo observar mensagens exibidas fora do DOM. Entradas numéricas fictícias exigem um perfil descartável local explícito. Seus valores respeitam os limites e o incremento declarados no HTML; credenciais continuam sujeitas à política de acesso.

O histórico preserva etapas, erros, testes, capturas e hipóteses em `run-log.json` e artefatos associados. A interface permite registrar revisões humanas com justificativa e conservar as decisões anteriores. Até 20 revisões recentes por caminho de projeto podem contextualizar os papéis autor e crítico. Essa recuperação de contexto complementa a evidência da execução corrente, sem atualizar os pesos do modelo.

### 4.5. Instrumentação para avaliação

O módulo de caracterização associa alvos a registros e arquivos, produzindo medidas estruturais, identificação dos serviços e referências às asserções. Comportamentos de referência anotados pelo avaliador permanecem separados do resultado automático. O suplemento publica protocolos, trajetórias e verificações selecionadas, acompanhados de scripts para reproduzir os ensaios. Esse material permite conferir cada conclusão no nível da aplicação, ação e propriedade avaliada.

## 5. Resultados e Discussão

### 5.1. Exploração inicial na instância atual

Foram registradas 14 explorações em sete aplicações no Windows e oito explorações adicionais em quatro desses alvos no Apple M4. A Tabela 2 apresenta a rodada inicial no Windows. As contagens incluem ações concluídas; os efeitos permitem interpretar seu alcance.

**Tabela 2. Explorações da instância atual: ações concluídas e estados por repetição.**

| | Ações | | Estados | |
|---|---|---|---|---|
| Aplicação | R1 | R2 | R1 | R2 |
| Expense Tracker | 5 | 5 | 4 | 4 |
| Movie Seat Booking | 17 | 17 | 18 | 18 |
| Modal Menu Slider | 8 | 8 | 4 | 4 |

As seis explorações registraram 60 ações concluídas e 52 estados somados entre as tentativas. Os identificadores distinguem observações dentro de cada execução. A conferência dos arquivos confirmou que os serviços permaneceram inalterados durante a coleta.

Nas duas explorações do registro financeiro, o modelo preencheu a descrição *Lunch*, informou 15,00, confirmou o formulário e recarregou a página. O histórico manteve o lançamento e o saldo observado foi 15,00. A trajetória alcançou, assim, criação e restauração de estado de domínio. O catálogo informou os limites numéricos e o incremento aceito pelo campo, permitindo que o modelo escolhesse um valor sintético compatível com a validação nativa.

No seletor de assentos, a trajetória percorreu escolhas de filme e lugares, produzindo a maior quantidade de estados da rodada. No exemplo de menu e diálogo, percorreu o ciclo de abertura, interação com campos e fechamento dos painéis. A comparação evidencia alcances distintos: criação e restauração de um lançamento, seleção entre controles e transições de apresentação. A finalidade da aplicação permite interpretar os passos como operações de uso.

### 5.2. Exploração em contextos complementares

A Tabela 3 compara as operações observadas nos quatro alvos compartilhados. O número de passos descreve atividade; a descrição por função mostra o que cada trajetória alcançou.

No TodoMVC, a tarefa inserida aparece após Enter e fica fora do filtro Completed; nenhuma trajetória conclui a tarefa. No Memory Cards, a criação não foi concluída e a recarga ocorreu com a coleção vazia. A regra automática da matriz creditou restauração por aceitar uma recarga após qualquer mudança anterior, inclusive de um campo de entrada. A inspeção dos passos mostrou que não havia cartão a restaurar. A matriz foi útil para localizar essa divergência, e a classificação funcional decorre do efeito observado.

No formulário, a política restringiu e-mail e senha. As hipóteses incluíram validação imediata de Username e ausência de submissão ao pressionar Enter. A conferência

**Tabela 3. Ações concluídas por repetição (R1/R2) e alcance nos quatro alvos compartilhados.**

| Aplicação | Windows 27B | M4 4B | Alcance observado |
|---|---|---|---|
| Form Validator | 2/2 | 2/2 | Preenchimento de Username e submissão por Enter nas duas configurações. |
| Music Player | 3/3 | 2/2 | Faixa anterior e exceção nos dois perfis; ação de progresso indisponível no M4. |
| Memory Cards | 9/9 | 6/8 | Campos preenchidos sem criação de cartão. Navegação vazia executada no Windows. |
| TodoMVC React | 5/5 | 5/5 | Criação de tarefa e uso de filtros nas duas configurações. |

mostrou que digitar atualiza o valor, sem validar antes da submissão. Enter e Submit disparam o mesmo evento, marcam Username como válido e exibem as mesmas mensagens nos demais campos. A inspeção do manipulador de submissão sustenta essa leitura: os efeitos foram reproduzidos, mas as expectativas que os classificavam como defeitos não tinham fundamento.

### 5.3. Validação das propriedades observadas

A suíte interna registrada na coleta original contém 152 testes aprovados, incluindo contratos de decisão, identidade de campo, correspondência entre ação e estado e coerência do crítico. Esses testes avaliam o instrumento; as verificações seguintes tratam das propriedades dos alvos.

A Tabela 4 sintetiza quatro propriedades verificadas com a mesma asserção em versões de referência e cópias alteradas. O teste aceita o comportamento esperado e falha por asserção quando a propriedade é violada. No registro financeiro, os testes de saldo e persistência são independentes: desabilitar uma dessas propriedades não faz o outro teste falhar.

**Tabela 4. Discriminação de propriedades pela mesma asserção na referência e na cópia alterada.**

| Propriedade | Diferença introduzida na cópia | Referência | Alterada |
|---|---|---|---|
| Identidade da faixa | Desativar a ação de faixa anterior | Passa | Falha |
| Saldo do lançamento | Desativar a atualização do saldo | Passa | Falha |
| Lançamento após recarga | Desativar o armazenamento local | Passa | Falha |
| Valor aprovado no critério | Produzir 42 quando o requisito exige 99 | Passa | Falha |

O formulário de controle verifica uma propriedade central do contrato. Mesmo quando a trajetória registrada mostra 42, o critério aprovado exige 99; o teste gerado conserva 99 como resultado esperado. A implementação de referência aplica essa regra e passa, enquanto a cópia que mantém 42 falha. Esse procedimento confirma que a observação do alvo não substitui automaticamente a expectativa utilizada para avaliá-lo.

No ensaio de geração e execução do Memory Cards no M4, os três testes gerados passaram. A inspeção confirmou asserções sobre valores digitados e um título, sem verificar as funções de referência dos cartões. Em uma conferência posterior no Windows, a mesma suíte passou tanto no original quanto em uma cópia com a criação desativada. Um roteiro de controle, elaborado para a conferência, criou e restaurou um cartão no original e constatou sua ausência na cópia. A aprovação da suíte indicou que a interface exercitada respondia; a comparação mostrou que ela não detectava a perda da criação.

### 5.4. Observação e reprodução de falhas

Nas duas configurações, a instrumentação registrou no Music Player a exceção de escrita em elemento ausente durante a atualização do tempo (`currTime` e `durTime`). A conferência no navegador constatou mudança da faixa e funcionamento do comando de pausa, mesmo com a exceção. O diagnóstico, portanto, localiza uma falha na atualização de tempo, sem sustentar as alegações sobre falha de troca de faixa ou do botão de reprodução. No Memory Cards, a navegação sem cartão atingida no Windows gera uma exceção ao acessar `cardsEl[currentActiveCard]`; nenhuma ação de navegação foi concluída nas explorações do M4.

O modelo selecionou as ações que alcançaram os estados defeituosos; o coletor identificou as exceções; a reprodução e a inspeção relacionaram trajetória, diagnóstico e causa. Os formulários de rotulação e a rotina de cálculo de concordância preparam a revisão das hipóteses por pessoas. Os formulários ainda precisam incorporar os fatos e as expectativas dos registros, e não houve dupla rotulação humana neste estudo.

**Verificação após a coleta.** Após a rodada no Windows, o reprodutor foi ajustado para tratar recarga e espera como operações do navegador. A conferência então repetiu as quatro trajetórias com diagnóstico em sessões limpas. O suplemento conserva as explorações originais separadas dessas reproduções, que reutilizam passos gravados.

### 5.5. Profundidade exploratória e capacidade computacional

A coleta no M4 demonstra viabilidade em uma segunda configuração local, com operações comuns e diferenças de alcance nos controles de mídia e cartões. Hardware, modelo, quantização, contexto e ambiente de execução variaram juntos. A unidade de comparação é a configuração completa. Mais memória pode permitir menor compressão dos pesos ou mais evidência por chamada; menor latência pode permitir mais decisões no mesmo intervalo. Medir esses efeitos separadamente requer registrar tokens utilizados, memória, latência e funções alcançadas enquanto se varia um fator por vez.

### 5.6. Respostas às questões de pesquisa

Para a **QP1**, as trajetórias exercitaram operações de entrada, persistência e filtragem, com alcance parcial por alvo. Para a **QP2**, a instrumentação capturou exceções reproduzíveis; as hipóteses de formulário exigiram conferência separada do comportamento esperado. Para a **QP3**, as asserções específicas preservaram a expectativa e discriminaram alterações, enquanto o caso dos cartões mostrou uma suíte aprovada sem essa capacidade. Para a **QP4**, a execução foi viável nos dois ambientes, com diferenças observadas no alcance. O efeito isolado de memória, contexto ou modelo permanece aberto.

## 6. Ameaças à Validade

**Construção.** A matriz cobre funções selecionadas, definidas por quem conhecia as coletas anteriores. Suas regras de associação podem omitir uma ação indisponível sem identificador ou creditar um efeito que a trajetória não demonstra. O reconhecimento textual de asserções também pode aceitar uma expressão alheia à função. A análise dos casos usa evidência da trajetória e do teste; as contagens não estimam cobertura funcional completa. As alterações controladas avaliam asserções sobre jornadas conhecidas.

**Validade interna.** Cada rodada usa o mesmo perfil em suas repetições, com temperatura zero nas decisões estruturadas. Rede, temporização e dados externos podem afetar o resultado. A pausa do relógio modifica a temporalidade de uso. A revisão das trajetórias foi realizada separadamente das explorações e não reproduziu a inferência em Apple Silicon.

**Avaliação e atribuição.** A análise dos candidatos foi assistida por IA e inspeção dos artefatos. Autor e crítico compartilham o perfil de modelo, e a concordância humana ainda não foi medida. Sem um conjunto exaustivo de defeitos e uma linha de base sob o mesmo orçamento, não se estima precisão, revocação ou vantagem sobre outra estratégia. O efeito da recuperação de revisões anteriores sobre falsos positivos não foi medido.

**Validade externa.** Predominam aplicações didáticas de uma origem comum, acrescidas de uma aplicação React, em modo convidado e Chromium. O porte delimita a transferência para sistemas com autenticação e persistência de servidor. Os dois perfis locais foram observados em máquinas diferentes, sem comparação isolada de modelo. Não se verificou se os exemplos fizeram parte do treinamento: “desconhecida” designa a apresentação inicial à sessão.

## 7. Conclusões e Trabalhos Futuros

A contribuição do E2P é conectar a iniciativa exploratória de modelos locais a ações executáveis e a evidências que sustentam sua verificação. Nos dois ambientes, as trajetórias iniciaram operações de uso sem roteiro e alcançaram uma exceção reproduzível em comum. Os ensaios mostraram tanto asserções que discriminam alterações quanto uma suíte aprovada que não detectou a perda da criação de cartões. O instrumento torna examinável a relação entre a escolha da ação, o resultado observado e o critério avaliado.

Em relação aos agentes web discutidos na literatura, o trabalho examina a iniciativa anterior à especificação de uma tarefa; em relação à geração de testes, acompanha a origem e a conservação do oráculo. O caso do formulário evidencia por que explorar, reproduzir e julgar uma violação são atividades complementares. A resposta ao problema de pesquisa é, assim, uma capacidade inicial observada em operações delimitadas, apoiada por mecanismos que permitem conferir os resultados e recusar expectativas sem fundamento.

O próximo passo experimental é comparar as decisões do modelo com seletores aleatórios ou orientados à novidade, mantendo catálogo, orçamento e compilador. A matriz deve explicitar efeitos por função e vincular cada asserção à entidade e à ação que a fundamentam. A revisão independente das hipóteses e novos ensaios com alterações controladas complementariam essa comparação.

Como continuidade, propõe-se utilizar o **JEV**, da TypeSafe, como componente

de decisão e avaliação estruturada [TypeSafe AI 2026]. O modelo recebe estado e perguntas tipadas, retornando respostas e distribuições de probabilidade. No E2P, pode ser investigado para selecionar entre ações catalogadas, avaliar se uma evidência sustenta determinada hipótese e encaminhar casos incertos à revisão humana. A avaliação dessa integração deve comparar concordância com julgamentos humanos, custo, latência e comportamento em casos ambíguos, com limiares definidos para cada decisão.

Outra direção é comparar máquinas e configurações de inferência conforme o procedimento discutido na Seção 5.5, medindo funções alcançadas, estabilidade e evidências de falha. Aplicações com jornadas integradas, diferentes perfis de acesso e persistência no servidor ampliariam os comportamentos investigados.

## Declaração de uso de inteligência artificial

A ferramenta Codex, da OpenAI, foi utilizada na revisão e atualização das diferentes seções deste artigo: organização textual, tradução do resumo, conferência bibliográfica, confronto com código e artefatos experimentais e formatação LaTeX. Registros de implementação e avaliação também incluem operações assistidas por IA. A definição do problema e das questões de pesquisa, as escolhas metodológicas e a interpretação das evidências integram a contribuição intelectual pela qual o autor responde; a ferramenta oferece sugestões e realiza operações de apoio nessas atividades.

## Referências


Ahmed, S. Y., Feng, S., Bae, C., Barrus, C., and Zhang, X. (2026). SpecOps: A fully automated AI agent testing framework in real-world GUI environments. In *Proceedings of the 48th International Conference on Software Engineering (ICSE)*. https://doi.org/10.1145/3744916.3787778.

da Cunha Franco, H. O. (2026). *Dopa*. Repositório de software. https://github.com/pieceofhell/dopa. Cópia local utilizada na avaliação de 8 set. 2026.

Drouin, A., Gasse, M., Caccia, M., Laradji, I. H., Del Verme, M., Marty, T., Boisvert, L., Thakkar, M., Cappart, Q., Vazquez, D., Chapados, N., and Lacoste, A. (2024). WorkArena: How capable are web agents at solving common knowledge work tasks? arXiv:2403.07718, versão 5. https://arxiv.org/abs/2403.07718.

Ferreira, M., Viegas, L., Faria, J. P., and Lima, B. (2025). Acceptance test generation with large language models: An industrial case study. In *IEEE/ACM International Conference on Automation of Software Test (AST)*. https://doi.org/10.1109/AST66626.2025.00007.

ggml-org (2026). *llama.cpp*. Software. https://github.com/ggml-org/llama.cpp. Acesso em: 13 set. 2026.

Gu, S., Nashid, N., and Mesbah, A. (2026). LLM test generation via iterative hybrid program analysis. In *Proceedings of the 48th International Conference on Software Engineering (ICSE)*. https://doi.org/10.1145/3744916.3764553.

Kang, S., Yoon, J., and Yoo, S. (2023). Large language models are few-shot testers: Exploring LLM-based general bug reproduction. In *Proceedings of the 45th International Conference on Software Engineering (ICSE)*. https://arxiv.org/abs/2209.11515.

Kolthoff, K., Kretzer, F., Ponzetto, S. P., Maedche, A., and Bartelt, C. (2025). GUISpector: An MLLM agent framework for automated verification of natural language requirements in GUI prototypes. arXiv:2510.04791. https://arxiv.org/abs/2510.04791.

Le, C. C., Van, C. D., Vu, T. D., Vu, T. M. P., Phan, H. N., Phan, H. N., and Nguyen, T. N. (2026). TestWeaver: Execution-aware, feedback-driven regression testing generation with large language models. In *Proceedings of the 48th International Conference on Software Engineering (ICSE)*. https://arxiv.org/abs/2508.01255.

Liu, Z., Chen, C., Wang, J., Chen, M., Wu, B., Che, X., Wang, D., and Wang, Q. (2024). Make LLM a testing expert: Bringing human-like interaction to mobile GUI testing via functionality-aware decisions. In *Proceedings of the 46th International Conference on Software Engineering (ICSE)*. https://arxiv.org/abs/2310.15780.

Microsoft (2026). *Playwright: Assertions*. Documentação. https://playwright.dev/docs/test-assertions. Acesso em: 13 set. 2026.

TasteJS (2026). *TodoMVC: React example*. Repositório de software. https://github.com/tastejs/todomvc/tree/master/examples/react. Cópia local identificada no suplemento; acesso em: 20 out. 2026.

Traversy, B. (2026). *Vanilla Web Projects*. Repositório de exemplos. https://github.com/bradtraversy/vanillawebprojects. Cópia local utilizada nas avaliações de set. 2026.

TypeSafe AI (2026). *Jev: Introduction*. Documentação oficial. https://docs.typesafe.ai/introduction. Acesso em: 18 out. 2026.

Wang, J., Huang, Y., Chen, C., Liu, Z., Wang, S., and Wang, Q. (2024). Software testing with large language models: Survey, landscape, and vision. *IEEE Transactions on Software Engineering*. Versão dos autores: https://arxiv.org/abs/2307.07221.

Zhao, Y., Talebipour, S., Baral, K., Park, H., Yee, L., Khan, S. A., Brun, Y., Medvidović, N., and Moran, K. (2022). Avgust: Automating usage-based test generation from videos of app executions. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*. https://arxiv.org/abs/2209.02577.

Zhou, S., Xu, F. F., Zhu, H., Zhou, X., Lo, R., Sridhar, A., Cheng, X., Ou, T., Bisk, Y., Fried, D., Alon, U., and Neubig, G. (2024). WebArena: A realistic web environment for building autonomous agents. In *International Conference on Learning Representations (ICLR)*. https://arxiv.org/abs/2307.13854.